

Clase Práctica Semana 2 - Generación y Manejo de Vectores (Arrays)

Se realizará un proyecto incremental para introducir y consolidar **La creación y administración de vectores** iniciando de un array numerico simple para luego implementar un array de referencias de un tipo llamado TarjetaBancaria

La propuesta parte de una implementación mínima y evoluciona progresivamente. Cada fase agrega comportamiento, introduce nuevos conceptos y conserva los tests de las fases anteriores como red de seguridad.

El objetivo no es resolver el problema completo desde el comienzo, sino **hacer crecer el modelo mientras aparecen naturalmente los conceptos de diseño que necesitamos discutir**.

Enunciado General - Administración Cartera de Clientes en Banco

Introducción

El departamento de Riesgo Crediticio de un banco necesita analizar la distribución de sus clientes actuales. Cada **Cliente** tiene asignado un Nivel Crediticio numérico (0 Nivel Moroso, 1 para Bronce, 2 para Plata y 3 para Oro) según su comportamiento financiero.

El banco requiere generar un reporte estadístico para saber exactamente cuántos clientes pertenecen a cada nivel.

Cada cliente posee un **codigo**, un **nombre**, el **saldo** que ese cliente posee en el banco y su **nivel** crediticio que se toma los valores indicados anteriormente.

La solución será construida incrementalmente durante las distintas fases del ejercicio.

Estado del Cliente

Codigo

Cada cliente tendrá un valor de identificación unico que se asignará en la creación del cliente y sera único

```
// Generación del código
UUID guid = UUID.randomUUID();
String guidString = guid.toString();
```

Nombre

Un valor de cadena que es el nombre de la persona.

Saldo

Un valor double que representa el dinero que posee en el Banco, el valor puede ser negativo.

Nivel

Un valor entre 0 y 3 que representa el nivel detallado anteriormente.

```
public enum NivelCrediticio {  
    MOROSO,    // ordinal 0  
    BRONCE,    // ordinal 1  
    PLATA,     // ordinal 2  
    ORO        // ordinal 3  
}
```

Comportamientos

Modificar Saldo

Que agrega o decrementa el saldo del cliente dependiendo del signo del valor que se envia

Actualizar Nivel Crediticio

Se asigna el nivel en base al saldo cuando este se haya actualizado, con las siguientes reglas:

- Si es un valor negativo el nivel es **MOROSO**
- Si es un valor mayor a cero pero menor a 300000 **BRONCE**
- Si es un valor mayor igual a 300000 pero menor a 1000000 **PLATA**
- Si es mayor al millon **ORO**

Fase I - Cración de Clase Cartera

Esta clase contendrá un unico atributo, un arreglo que contendrá los datos de las **cuentas**, en principio de tipo Object

Comportamientos

- Por defecto se pueden tener 10 cuentas
- Por valor establecido se pueden tener **N** cuentas donde N es mayor a cero
- Agregar cuentas a la cartera
- Buscar una cuenta
- Obtener tamaño del arreglo
- Obtener un elemento **i** de la cartera

Constructor por defecto

Establece un cantidad de 10 elementos en el arreglo cuentas

Constructor con un size

Establece una cantidad de n elementos en el arreglo cuentas

Agregar una cuenta

Se le agrega a la arreglo en la primera posicion not null del arreglo, si no hay lugar disponible lanzar `IllegalStateException` -> *"Error: El arreglo está lleno. Capacidad máxima alcanzada."*

Buscar una cuenta

A partir de un codigo de cuenta, buscar y si existe ese elemento retornar la posicion donde esa cuenta se encuentra. Si no existe lanzar la excepcion `NoSuchElementException` -> *"Error: NO existe la cuenta buscada en el arreglo."*

Obtener tamaño

Retorna el length del arreglo

Obtener un elemento *i*

Retorna la cuenta de la posición *i* del arreglo

Fase II - Upgrade de Clase Cartera

Ahora agregar a la clase Cartera la posibilidad de que el atributo **cuentas** no sea del tipo Object sino Cuenta, esto debera modificar todos los metodos para que manejen dicho tipo. Tambien agregar un atributo **posicion** que determine la cantidad de elementos ocupados del arreglo, al momento de crear el objeto Cartera **este valor es 0**

Comportamientos

- Asegurar capacidad
- Eliminar elemento
- Mantener Integridad

Asegurar capacidad

Si se desea agregar una nueva cuenta y no hay posicion libres

```
posicion >= this.cuentas.length
```

Debe:

- Crear un nuevo arreglo con el doble de la capacidad del existente

- Copiar los elementos existentes al nuevo array

```
System.arraycopy(this.items, 0, nuevo, 0, this.items.length);
```

- Asignar el nuevo al atributo **cuentas**

Eliminar elemento

Quita un elemento del arreglo en base a un posicion que se envia, debe lanzar NoSuchElementException -> "Remove: No existe el elemento a eliminar"

Tener en cuenta:

- Si existe asignar a una variable el elemento a borrar
- Quitarlo manteniendo la integridad
- Se devuelve ese elemento

Mantener Integridad

Al eliminar un elemento, se debe redimensionar con un nuevo array los elementos a la izquierda y a la derecha de la posicion eliminada y asinar ese nuevo array al atributos **cuentas**

```
System.arraycopy(this.items, 0, nuevo, 0, pos);
```

Fase III - Conteo de Frecuencia

El objetivo es determinar, a partir de las cuentas que se encuentran contenidas en cartera, cuantas de ellas hay para cada tipo de **Nivel** se tiene.

Para ello use esta:

```
public class ReporteNivel {  
    private final NivelCrediticio nivel; // El nivel no cambia  
    private int cantidad;                // La cantidad SÍ puede cambiar  
  
    // El método clave: modifica el atributo interno directamente sin usar 'new'  
    public void incrementar() {  
        this.cantidad++;  
    }  
}
```

Pista: la posicion 0 deberia ser MOROSO, 1 BRONCE, etc...

```
ReporteNivel[] frecuencia;  
frecuencia[0].incrementar();
```